Rapport de Réalisation :
Classification de Composants Électroniques par Intelligence Artificielle Embarquée

Projet TinyML

Table des matières

1 Introduction **2**

2 Matériel et Configuration **2**
2.1 Matériel utilisé 2
2.2 Outils logiciels 2

3 Constitution du Dataset **3**
3.1 Classes définies 3
3.2 Importance de la classe `unknown` 3
3.3 Upload des images sur Edge Impulse 3

4 Conception du Modèle (Impulse Design) **3**
4.1 Bloc d'entrée — Image 96×96 3
4.2 Bloc de traitement — Image Grayscale 3
4.3 Bloc d'apprentissage — Transfer Learning MobileNetV1 4

5 Entraînement et Résultats **4**
5.1 Paramètres d'entraînement 4
5.2 Résultats obtenus 4
5.3 Analyse des résultats 4

6 Déploiement sur Arduino **4**
6.1 Export de la librairie Edge Impulse 4
6.2 Sketch Arduino 5
6.3 Durée de téléversement 5
6.4 Configuration du port Serial dans Node-RED 5

7 Flow Node-RED et Dashboard **5**
7.1 Architecture du flow 5
7.2 Dashboard de comptage 6

8 Remarque : Simulation avec Webcam PC **6**

9 Limitations et Améliorations **7**

10 Conclusion **7**

1 Introduction

Ce projet explore les capacités de l'intelligence artificielle embarquée (TinyML) appliquée à la **vision par ordinateur**. L'objectif est de concevoir un système capable de reconnaître et de comptabiliser des composants électroniques en temps réel, directement sur un microcontrôleur.

Le système réalisé permet de :

— Capturer des images de composants électroniques via une caméra **OV7670**
— Classifier chaque composant grâce à un modèle d'IA entraîné sur **Edge Impulse**
— Transmettre les résultats à **Node-RED** via liaison série (USB)
— Afficher les compteurs sur un **dashboard** en temps réel

Ce rapport décrit la chaîne de traitement complète, de la constitution du dataset jusqu'au déploiement et la visualisation des résultats.

2 Matériel et Configuration

2.1 Matériel utilisé

Composant	**Rôle**
Arduino Nano 33 BLE Sense	Microcontrôleur principal (Cortex-M4, 256 KB RAM)
Caméra OV7670	Capture des images 160×120 pixels
Câble USB Micro-B	Communication Serial avec le PC (115200 baud)
PC avec Node-RED	Réception des données et dashboard de comptage

Table 1 – Matériel utilisé

2.2 Outils logiciels

Outil	**Usage**
Edge Impulse	Entraînement et export du modèle
Arduino IDE	Développement et téléversement du sketch
Librairie Arduino_OV767X	Pilote de la caméra OV7670
Librairie Edge Impulse (.zip)	Inférence TFLite embarquée
Node-RED v4.1.7	Flow de comptage et dashboard
Node.js v24.12.0	Environnement d'exécution Node-RED
Python 3.11 + OpenCV	Simulation via webcam PC

Table 2 – Outils logiciels utilisés

3 Constitution du Dataset

3.1 Classes définies

Classe	Description	Nb. images
`capacitor`	Condensateurs électrolytiques et céramiques	∼100
`diode`	Diodes de redressement	∼100
`resistor`	Résistances à couche de carbone	∼100
`transistor`	Transistors BJT (NPN/PNP)	∼100
`unknown`	Fond neutre, objets non reconnus	∼80
Total		**∼480**

Table 3 – Classes et distribution du dataset

3.2 Importance de la classe `unknown`

L'ajout d'une classe `unknown` est essentiel. Sans elle, le modèle force une classification même lorsqu'aucun composant reconnaissable n'est présent. Cette classe permet au système de répondre "je ne sais pas" plutôt que de produire une fausse détection.

3.3 Upload des images sur Edge Impulse

1. Les images ont été renommées au format `classe.N.jpg` pour l'inférence automatique du label
2. **Data Acquisition → Upload data → Select a folder**
3. Split automatique : **80% entraînement / 20% test**
4. Répété pour chacune des 5 classes

4 Conception du Modèle (Impulse Design)

4.1 Bloc d'entrée — Image 96×96

Paramètre	Valeur
Résolution	96 × 96 pixels
Mode de resize	Squash

Table 4 – Paramètres du bloc d'entrée

4.2 Bloc de traitement — Image Grayscale

Le traitement utilise une conversion en **niveaux de gris**, réduisant l'empreinte mémoire par rapport au mode RGB tout en conservant les informations structurelles nécessaires à la reconnaissance.

4.3 Bloc d'apprentissage — Transfer Learning MobileNetV1

Le modèle choisi est **MobileNetV1 96×96 0.1**, un réseau convolutif pré-entraîné optimisé pour les microcontrôleurs. Le *Transfer Learning* permet de réutiliser des poids pré-entraînés et de les affiner sur le dataset.

5 Entraînement et Résultats

5.1 Paramètres d'entraînement

Paramètre	**Valeur**
Training cycles	50
Learning rate	0.0001
Data augmentation	Activé
Modèle	MobileNetV1 96×96 0.1

Table 5 – Paramètres d'entraînement

5.2 Résultats obtenus

Classe	**Précision**
capacitor	82.4%
diode	76.5%
resistor	83.3%
transistor	55.0%
unknown	100%
Global	**76.9%**

Table 6 – Performances du modèle par classe

5.3 Analyse des résultats

- **Unknown (100%)** : Bonne performance.
- **Resistor (83.3%)** et **Capacitor (82.4%)** : Bonnes performances grâce à des formes visuellement distinctives.
- **Diode (76.5%)** : Performance acceptable.
- **Transistor (55.0%)** : Performance faible due à la ressemblance visuelle avec les condensateurs en niveaux de gris. Utiliser des images RGB améliorerait ce résultat.

6 Déploiement sur Arduino

6.1 Export de la librairie Edge Impulse

Le modèle a été exporté au format **Arduino library** (.zip) puis installé via **Sketch → Include Library → Add .ZIP Library**.

6.2 Sketch Arduino

Le sketch `arduino_camera_classification.ino` est basé sur l'exemple `nano_ble33_sense_camera` fourni par Edge Impulse. Il a été modifié pour envoyer les résultats vers Node-RED via Serial au format :

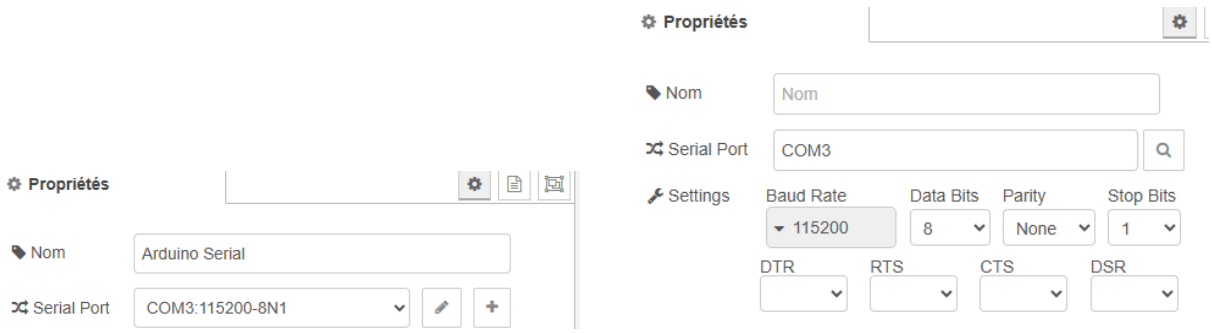
`classe,confiance` exemple : `capacitor,0.87`

Seules les détections avec une confiance supérieure à **60%** sont transmises. La classe `unknown` est ignorée.

6.3 Durée de téléversement

Le téléversement du sketch est **particulièrement long** (10 à 15 minutes), car il embarque le modèle TFLite complet, le pilote OV7670 et les fonctions d'inférence. Il ne faut pas interrompre le processus et attendre le message *"Upload complete"* dans Arduino IDE.

6.4 Configuration du port Serial dans Node-RED



(a) Nom du node et port COM3 (b) Configuration : 115200 baud, 8N1

FIGURE 1 – Configuration du port Serial dans Node-RED

7 Flow Node-RED et Dashboard

7.1 Architecture du flow

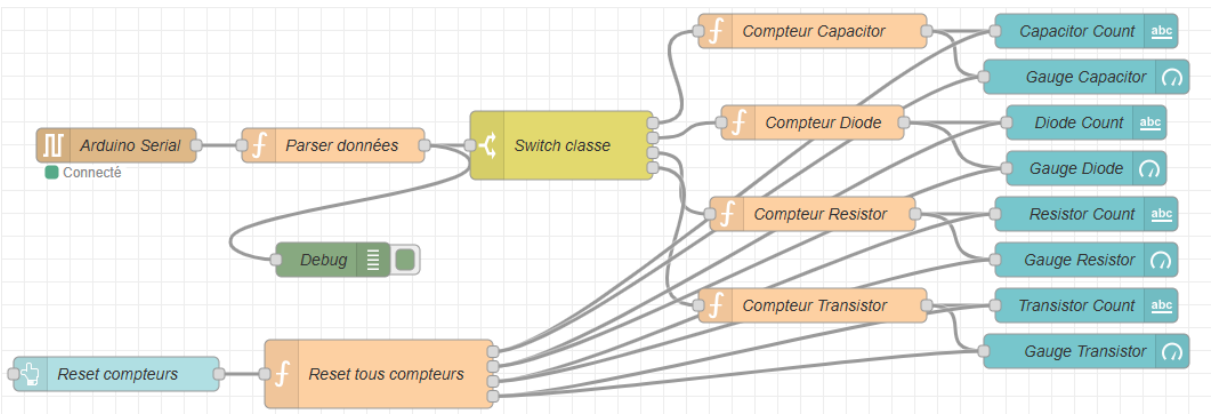


FIGURE 2 – Flow Node-RED : Serial → Parser → Switch → Compteurs → Dashboard

Le flow est composé des éléments suivants :

— **Arduino Serial** : Réception des données sur COM3 à 115200 baud
— **Parser données** : Extraction de la classe et de la confiance
— **Switch classe** : Aiguillage vers le bon compteur
— **Compteurs** : Incrémentation du total par classe
— **Jauges + Reset** : Affichage sur le dashboard avec remise à zéro

7.2 Dashboard de comptage

Le dashboard affiche en temps réel le nombre de composants détectés pour chaque classe, sous forme de jauges visuelles. La valeur maximale des jauges est fixée à 50 par défaut, mais elle peut être modifiée directement dans le flow Node-RED selon les besoins (par exemple, passer à 100 pour un comptage plus important).

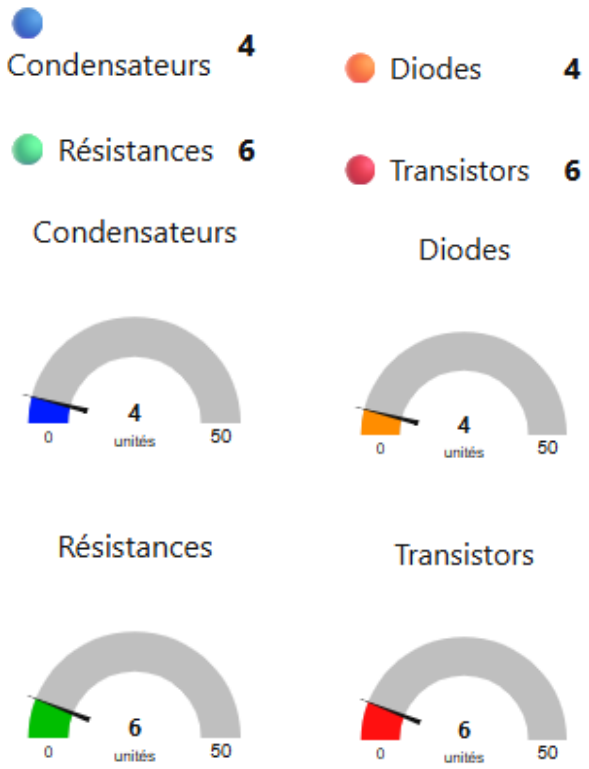


FIGURE 3 – Dashboard Node-RED — Comptage en temps réel des composants détectés

8 Remarque : Simulation avec Webcam PC

En raison de deux contraintes rencontrées lors du projet, une simulation avec la webcam du PC a été mise en place :

— La caméra OV7670 présente une **qualité d'image limitée** (résolution 160×120, absence de mise au point automatique), ce qui peut affecter la précision de la classification en conditions réelles.
— Le téléversement du sketch sur l'Arduino est **particulièrement long** (10 à 15 minutes), ce qui ralentit considérablement le cycle de développement et de test.

Pour pallier ces limitations, un script Python utilisant **OpenCV** a été développé. Il capture des images via la webcam du PC, les redimensionne en 96×96 pixels en niveaux de gris, puis envoie les résultats à Node-RED via une requête HTTP. Cette approche reproduit fidèlement le comportement du système réel et a permis de valider le bon fonctionnement du pipeline complet de la classification jusqu'à l'affichage sur le dashboard sans dépendre du matériel physique.

9 Limitations et Améliorations

- **Qualité de la caméra OV7670** : Résolution limitée et absence de mise au point automatique. Une caméra de meilleure qualité améliorerait la précision.
- **Précision du transistor (55%)** : Ressemblance visuelle avec les condensateurs en niveaux de gris. Passer en mode RGB permettrait de mieux les distinguer.
- **Taille du dataset** : ∼100 images par classe. L'ajout d'images avec angles et éclairages variés améliorerait la robustesse.
- **Durée de téléversement** : 10 à 15 minutes de compilation sur l'Arduino Nano 33 BLE Sense.

10 Conclusion

Ce projet a permis de valider la mise en œuvre d'un système complet de **vision par ordinateur embarquée**. En combinant le Transfer Learning avec MobileNetV1, la plateforme Edge Impulse et Node-RED, un système de classification de composants électroniques a été déployé directement sur un microcontrôleur Arduino.

La simulation réalisée avec la webcam PC a permis de valider l'ensemble du pipeline sans dépendre de la caméra OV7670, dont la qualité reste une limitation connue. Cette réalisation démontre l'efficacité du traitement local (*Edge Computing*) : les données sont analysées à la source, sans latence et sans dépendance au cloud.